

Creating a Moving Character with Animations

A practical Idle -> Walk -> Run locomotion workflow

UPDATED FOR THE NEW ANIMATION FEATURE

Includes full-body montage-style movement locking, masked/layered actions that keep locomotion active, the `IsAnimationMovementLocked()` script query, and the corrected Editor Play update path.

Build a controllable third-person character, assign skeletal clips, drive animation from movement speed, and add attacks or hit reactions without fighting the locomotion system.

Guide scope	Result
Editor workflow	Import, scene setup, Player Controller, locomotion clip assignment
C++ workflow	AnimatedModel, AnimationController::Locomotion, PlayerController, UpdateAnimations
Action workflow	Full-body locked actions and upper-body layered actions
Verification	Acceptance tests and symptom-based repairs

1. What You Will Build

The finished character stands in Idle without movement input, enters Walk for normal movement, and enters Run while sprinting. Movement is handled by PlayerController; animation is handled by AnimatedModel and its AnimationController.

Runtime relationship

```

Input -> PlayerController -> Transform and capsule movement
      |
      +--> movement speed -> AnimationController "Speed"

AnimationController -> Idle / Walk / Run base pose
One-shot action     -> full-body or masked pose overlay
UpdateAnimations    -> advances clips, blends pose, releases action lock
  
```

State thresholds

Speed	Base state	Typical meaning
Below Walk At	Idle	No meaningful movement
Walk At to Run At	Walk	Normal movement input
At or above Run At	Run	Sprint input or AI run speed

IMPORTANT

The speed parameter should describe intended horizontal movement, not model scale, camera distance, or imported animation playback speed.

2. Prepare the Character and Clips

Preferred editor asset

For the simplest editor workflow, use one FBX or glTF that contains the character skeleton and Idle, Walk, and Run clips. The clips must use the same skeleton and compatible bind pose.

Import checklist

- 1 Copy the character and textures into the project Content folder.
- 2 Import the skeletal model and inspect the clip list in the Animation panel.
- 3 Confirm Idle, Walk, and Run have valid durations and sensible names.
- 4 Verify the rig stands upright. For UE-style Z-up assets, use the engine's established -90 degree X correction.
- 5 Keep visual orientation correction separate from gameplay yaw so movement directions remain aligned.

Clip requirements

Clip	Loop	Root motion	Notes
Idle	Yes	Prefer in-place	Stable first and last frame
Walk	Yes	Prefer in-place	Forward cycle; feet contact the ground
Run	Yes	Prefer in-place	Faster cycle matching sprint speed
Attack / Hit	No	Project-dependent	Use as a one-shot action, not a locomotion loop

SKELETON MATCH

Separate animation files are safe only when they target the same hierarchy. A clip that loads but twists the mesh usually has a skeleton or bind-pose mismatch.

3. Create the Character in the Editor

- 1 Drag the skeletal model from Assets into the viewport and rename it Player.
- 2 Select Player and enable Skeletal Model in the Animation section.
- 3 Enable Use Locomotion.
- 4 Assign the Idle, Walk, and Run clip indices.
- 5 Start with Walk At = 0.15, Run At = 3.0, and Blend = 0.20 seconds.
- 6 Use the Animation preview to confirm every clip plays before entering Play mode.

Recommended scene setup

Item	Starting value
Player transform	On the ground, facing engine +Z
Player Controller	Third Person
Walk / Run speed	4.0 / 7.0
Capsule	Radius 0.4, height 1.8
Camera	Distance 5.0; tune height to the character
Animated model scale	Visual height matches the controller capsule

Orientation sanity test

- Press W: both the gameplay transform and the visible character should travel forward.
- Rotate the camera and press W: movement should remain camera relative.
- Use the model correction only to fix imported visual axes; do not rotate movement vectors by that correction.

4. Add Movement and Camera Control

PlayerController belongs on the same entity as AnimatedModel. It moves a kinematic capsule, updates the entity Transform, and supports third-person camera-relative input.

Editor controls

Input	Action
W / A / S / D	Move relative to camera
Shift	Sprint; selects run speed
Space	Jump when movement is enabled
Right mouse drag / pinned look	Rotate camera
V	Toggle first- and third-person view

Movement-disabled behavior

The controller's Update call now accepts movementEnabled. When false, translation and jumping are held, but camera look and gravity continue. This prevents a full-body attack from sliding while keeping the camera and falling behavior responsive.

```
const bool movementLocked = animated && animated->BlocksMovement();  
controller.Update(registry, input, dt, !movementLocked);
```

NO MANUAL TIMER REQUIRED

The animation action itself owns the lock lifetime. When the full-body clip completes, UpdateAnimations deactivates the action and movement resumes.

5. Build the Same Character in C++

Create the entity

```
engine::ecs::Entity player = registry.Create();

engine::ecs::Transform transform;
transform.position = glm::vec3(0.0f);
transform.scale = glm::vec3(1.0f);
registry.Add<engine::ecs::Transform>(player, transform);

engine::AnimatedModel animated;
animated.SetModel(characterModel);
animated.controller = engine::AnimationController::Locomotion(
    idleClip, walkClip, runClip,
    0.15f, 3.0f, 0.20f);
registry.Add<engine::AnimatedModel>(player, animated);
```

Configure the controller

```
engine::PlayerController controller;
controller.walkSpeed = 4.0f;
controller.runSpeed = 7.0f;
controller.jumpSpeed = 5.0f;
controller.radius = 0.4f;
controller.height = 1.8f;
```

Ownership rule

AnimatedModel stores per-entity controller state, action state, and the computed pose. The SkinnedModel pointer can be shared. Do not share one AnimatedModel instance between characters.

6. Drive Speed and Advance the Pose

Capture input

```
engine::PlayerInput input;  
input.moveForward = (keyW ? 1.0f : 0.0f) - (keyS ? 1.0f : 0.0f);  
input.moveRight  = (keyD ? 1.0f : 0.0f) - (keyA ? 1.0f : 0.0f);  
input.sprint     = shiftHeld;  
input.jump       = spacePressed;
```

Update in the correct order

```
auto* animated = registry.TryGet<engine::AnimatedModel>(player);  
const bool movementLocked = animated && animated->BlocksMovement();  
  
const bool moving = glm::length(glm::vec2(  
    input.moveRight, input.moveForward)) > 0.01f;  
const float speed = (!movementLocked && moving)  
    ? (input.sprint ? controller.runSpeed : controller.walkSpeed)  
    : 0.0f;  
  
if (animated) animated->controller.SetParameter("Speed", speed);  
controller.Update(registry, input, dt, !movementLocked);  
engine::UpdateAnimations(registry, dt);
```

FRAME ORDER

Choose actions and movement first, then call UpdateAnimations exactly once. This advances the pose and allows a completed full-body action to release its movement lock.

7. Full-Body Actions and Layered Actions

The new action behavior follows the useful part of montage workflows: whether an action blocks locomotion is determined by its bone mask.

Action	Mask	Movement	Examples
Full body	Empty	Blocked until completion	Heavy attack, hit, knockdown, death
Layered	Non-empty	Continues	Upper-body cast, aim, reload

C++ action calls

```
// Empty mask: full body and movement-blocking.
animated.PlayAction(heavyAttackClip);

// Non-empty per-bone mask: layered and movement remains enabled.
animated.PlayAction(castClip, upperBodyMask);

if (animated.BlocksMovement()) {
    // Optional ability or UI gate.
}
```

Native Script calls

```
PlayAnimationAction("HeavyAttack"); // blocks
PlayMaskedAnimationAction("StaffCast", "spine_01"); // moves

if (IsAnimationMovementLocked()) {
    // Suppress another action or show a lock indicator.
}
```

MASK ROOT

In an editor Action Profile, an empty Mask Root creates a full-body action. Choosing a spine bone creates a layered upper-body action. Use the latter when the character must keep walking while casting.

8. Editor Play and Custom Runtime Integration

Editor Play

Editor Play now performs the required integration automatically. It checks `BlocksMovement()` for the player, AI agents, and crowd displacement; passes `movementEnabled=false` to `PlayerController` during a full-body action; and calls `UpdateAnimations` in both the single-step and fixed-accumulator paths.

- A non-layered action stops player translation and jumping.
- A non-layered AI action stops steering and crowd displacement.
- A masked action does not stop locomotion.
- Camera look and gravity remain active while movement is locked.
- The lock ends when the one-shot action completes.

Custom runtime checklist

```
[ ] Read AnimatedModel::BlocksMovement before movement/AI steering
[ ] Set locomotion Speed to zero while movement is locked
[ ] Pass !movementLocked to PlayerController::Update
[ ] Skip AI steering and crowd displacement while locked
[ ] Call UpdateAnimations(registry, dt) exactly once per frame
[ ] Call UpdateAttachments after UpdateAnimations
```

COMMON INTEGRATION BUG

If `UpdateAnimations` is missing, the clip never reaches its end and a full-body action appears to lock forever.

9. Smooth the Locomotion

Cross-fade tuning

Symptom	Adjustment
Idle snaps into Walk	Increase locomotion blend from 0.20 toward 0.25-0.30
Run feels delayed	Reduce blend or lower Run At
Walk never appears	Run At is too low or supplied Speed jumps directly to run
Feet slide	Match controller speed to clip stride; do not solve only with blend time
Action pops on/off	Tune action Fade In and Fade Out independently

Threshold guidance

Walk At should be slightly above the noise floor. Run At should sit between the supplied walk and run speeds. For walkSpeed 4 and runSpeed 7, a Run At around 5 is semantically clearer than 3, although the engine default sample may use 3 for faster transitions.

Action tuning

- Use short fades for responsive melee attacks.
- Use a spine mask for casts that should preserve leg motion.
- Use an empty mask for hit reactions that must stop displacement.
- Keep one-shot clips non-looping and speed greater than zero.

10. Test and Debug

Acceptance test

- At rest, Idle loops and the pose continues updating.
- Normal input moves the capsule and selects Walk.
- Holding Shift selects Run without changing character scale.
- Releasing input returns smoothly to Idle.
- A full-body HeavyAttack freezes translation and jumping until the clip ends.
- Camera look and gravity still respond during HeavyAttack.
- A masked StaffCast plays on the upper body while the legs continue locomotion.
- All full-body attacks do not get pushed by steering or crowd displacement.

Symptom-based repairs

Symptom	Repair
Model faces down	Apply the imported-model -90 degree X correction; keep gameplay yaw separate.
Movement and gizmo disagree	Move using the entity transform, not the visual correction matrix.
Pose is frozen	Call UpdateAnimations once per frame and confirm clip indices are valid.
Full-body action never unlocks	Call UpdateAnimations; ensure action speed is above zero and the clip has duration.
Cast blocks movement unexpectedly	Assign an upper-spine Mask Root or call PlayMaskedAnimationAction.
Attack slides across the ground	Use an empty mask and honor BlocksMovement in the custom runtime.
Walk/run flickers	Separate thresholds and avoid a noisy Speed value around Run At.

DONE

The locomotion setup is complete when base movement blends cleanly, full-body actions lock exactly for their clip lifetime, and masked actions preserve movement without disturbing the lower-body cycle.